

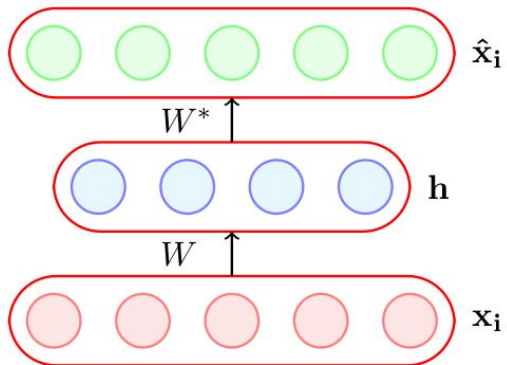
CS 564: Auto Encoder

Asif Ekbal

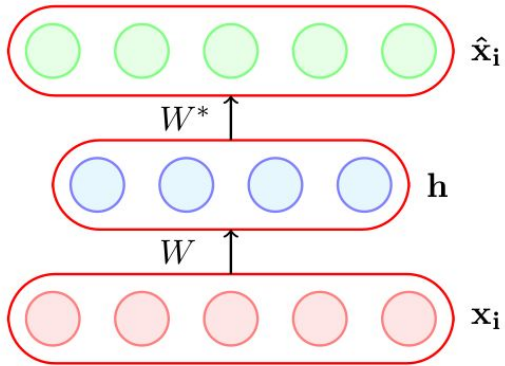
Dept of CSE

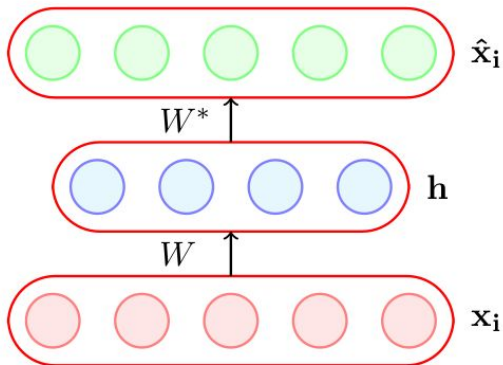
Indian Institute of Technology Patna

Introduction to Autoencoders

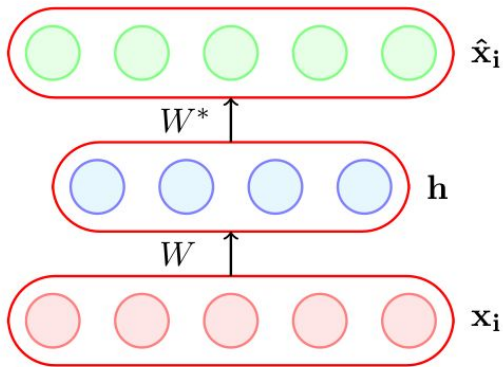


- An autoencoder is a special type of feed forward neural network which does the following



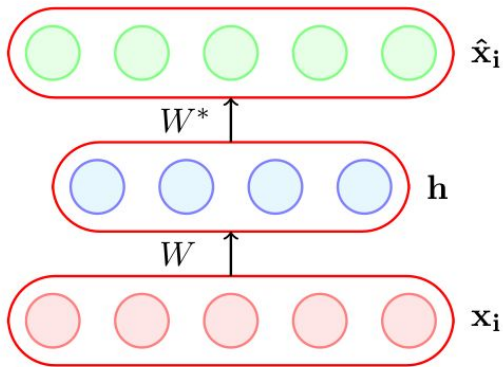


- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input $\mathbf{x}_i$ into a hidden representation $\mathbf{h}$



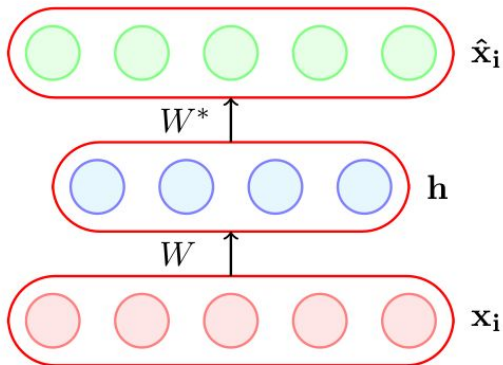
$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input $\mathbf{x}_i$ into a hidden representation $\mathbf{h}$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

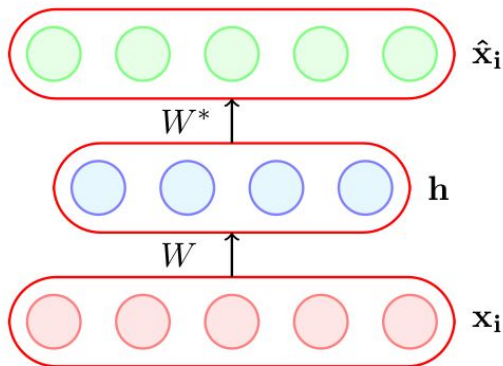
- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input $\mathbf{x}_i$ into a hidden representation $\mathbf{h}$
- Decodes the input again from this hidden representation



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

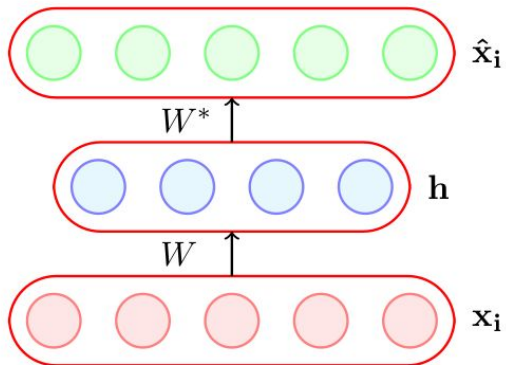
- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input $\mathbf{x}_i$ into a hidden representation $\mathbf{h}$
- Decodes the input again from this hidden representation



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

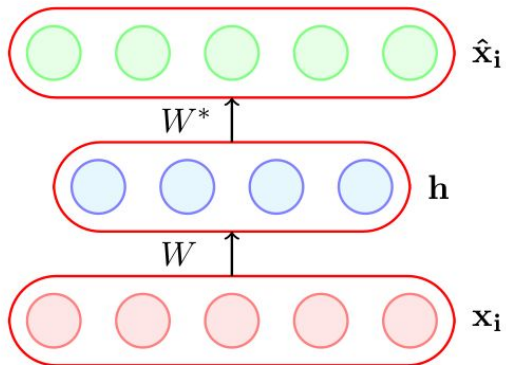
$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input $\mathbf{x}_i$ into a hidden representation $\mathbf{h}$
- Decodes the input again from this hidden representation
- The model is trained to minimize a certain loss function which will ensure that $\hat{\mathbf{x}}_i$ is close to $\mathbf{x}_i$ (we will see some such loss functions soon)



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

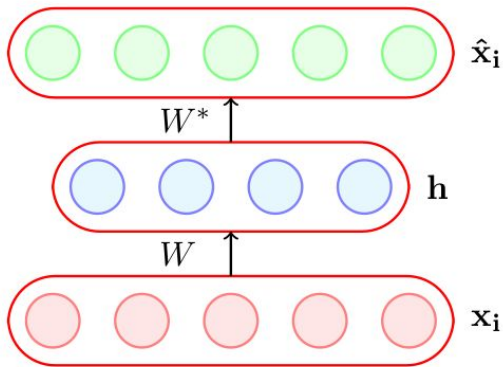
$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

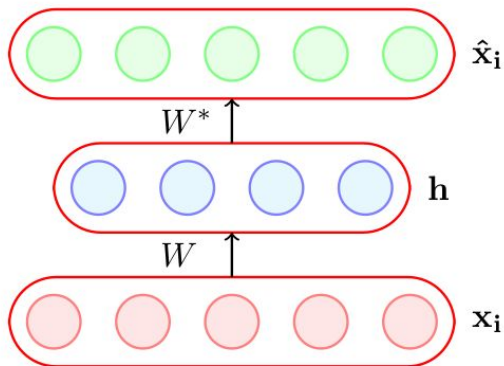
- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

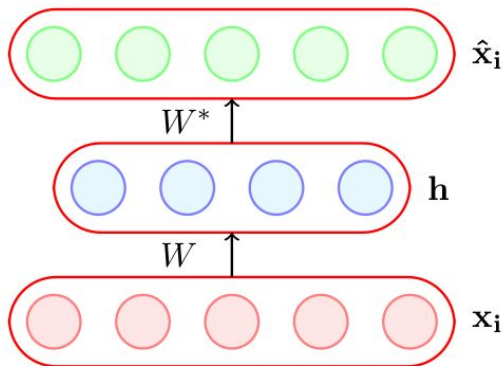
- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$
- If we are still able to reconstruct $\hat{\mathbf{x}}_i$ perfectly from $\mathbf{h}$, then what does it say about $\mathbf{h}$?



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

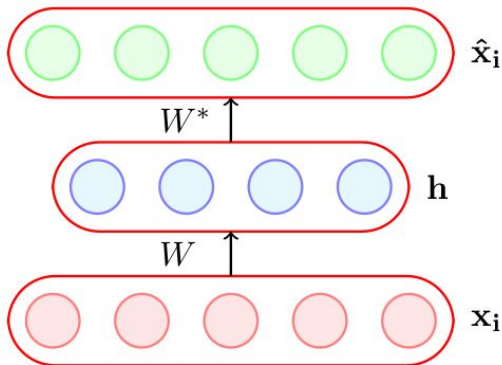
- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$
- If we are still able to reconstruct $\hat{\mathbf{x}}_i$ perfectly from $\mathbf{h}$, then what does it say about $\mathbf{h}$?
- $\mathbf{h}$ is a loss-free encoding of $\mathbf{x}_i$. It captures all the important characteristics of $\mathbf{x}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$
- If we are still able to reconstruct $\hat{\mathbf{x}}_i$ perfectly from $\mathbf{h}$, then what does it say about $\mathbf{h}$?
- $\mathbf{h}$ is a loss-free encoding of $\mathbf{x}_i$. It captures all the important characteristics of $\mathbf{x}_i$
- Do you see an analogy with PCA?

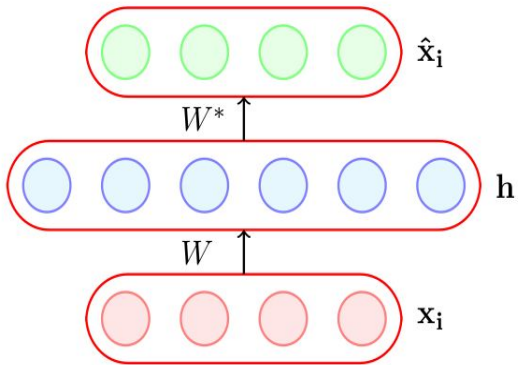


$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

An autoencoder where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$ is called an under complete autoencoder

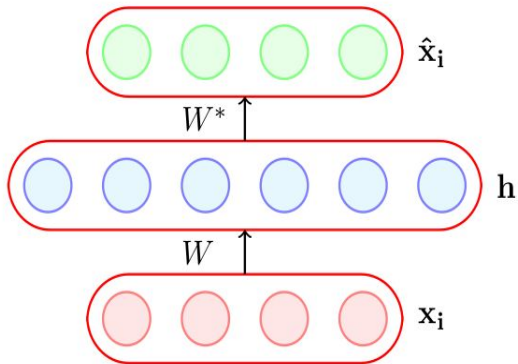
- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$
- If we are still able to reconstruct $\hat{\mathbf{x}}_i$ perfectly from $\mathbf{h}$, then what does it say about $\mathbf{h}$?
- $\mathbf{h}$ is a loss-free encoding of $\mathbf{x}_i$. It captures all the important characteristics of $\mathbf{x}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

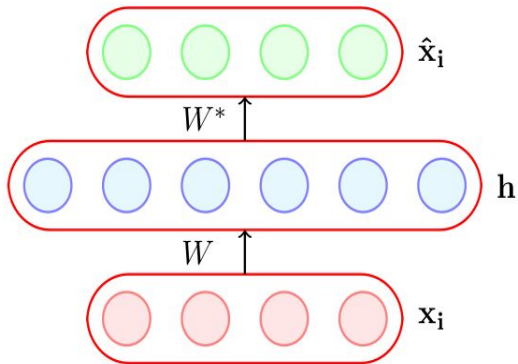
$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

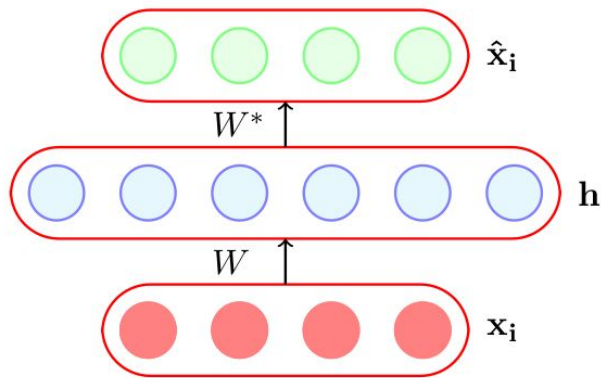
$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

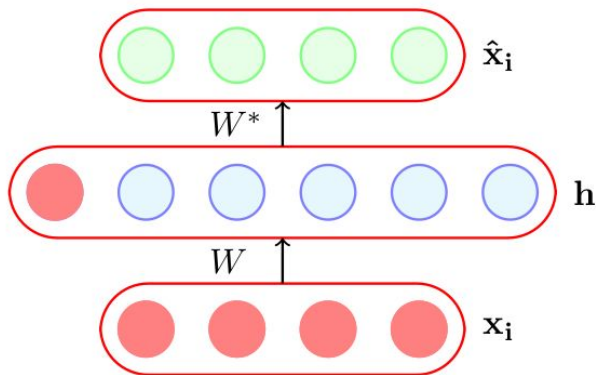
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

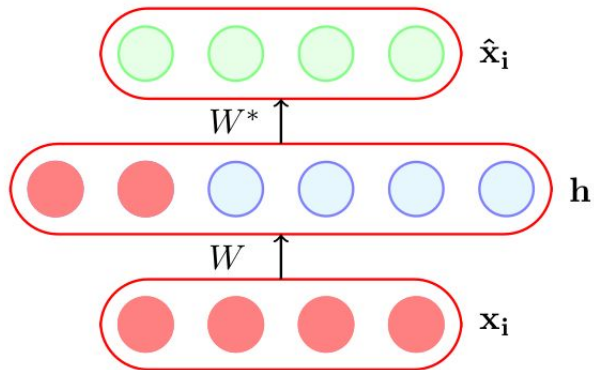
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

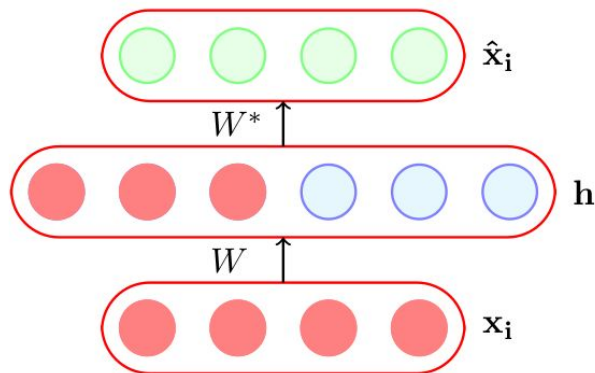
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

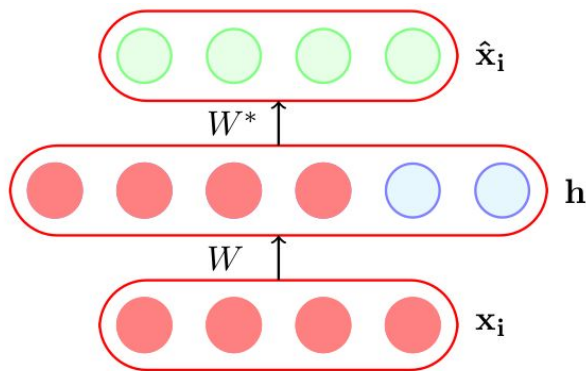
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

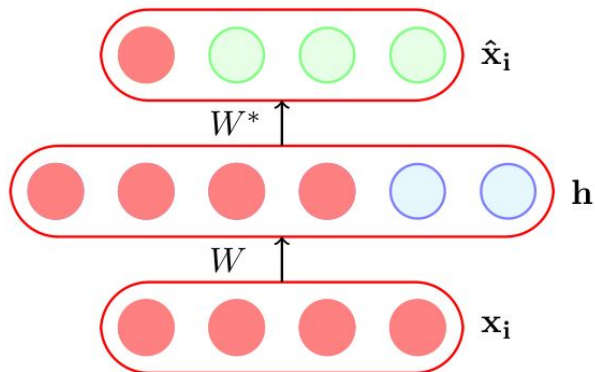
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

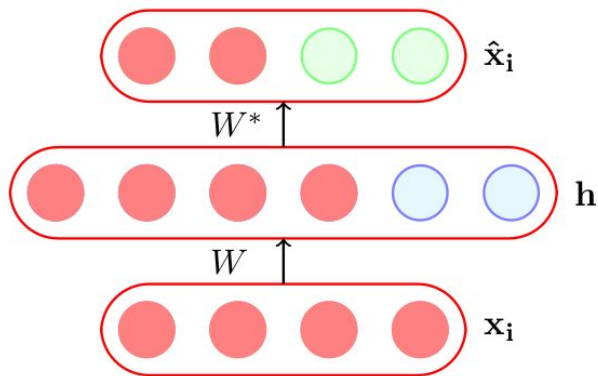
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

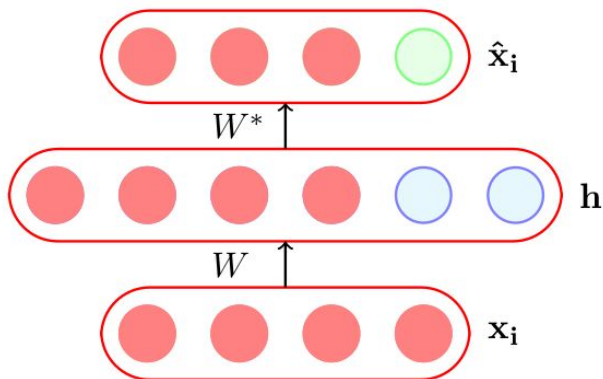
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

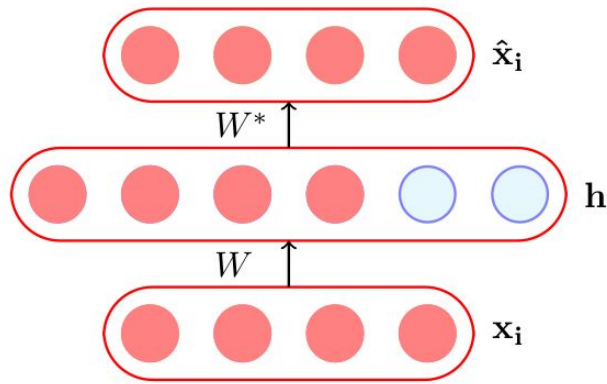
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

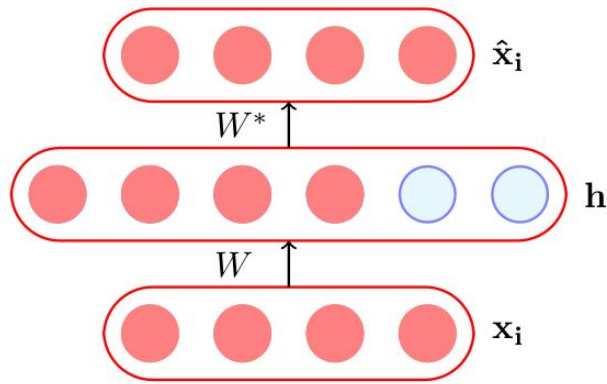
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

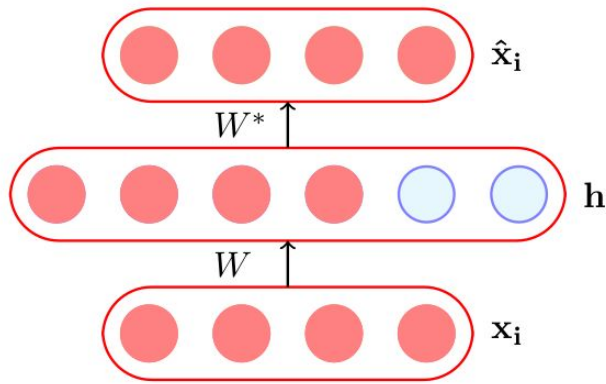
- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$
- Such an identity encoding is useless in practice as it does not really tell us anything about the important characteristics of the data.



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Let us consider the case when $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$
- Such an identity encoding is useless in practice as it does not really tell us anything about the important characteristics of the data.

An autoencoder where $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$ is called an **over-complete** autoencoder

The Road Ahead

The Road Ahead

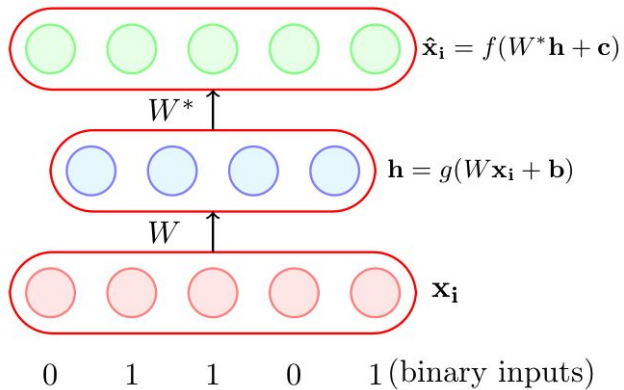
- Choice of $f(\mathbf{x}_i)$ and $g(\mathbf{x}_i)$

The Road Ahead

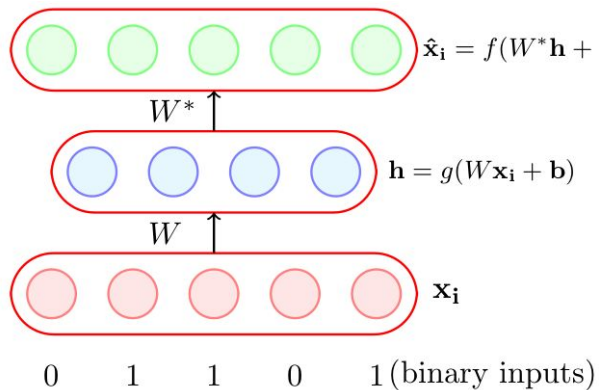
- Choice of $f(\mathbf{x}_i)$ and $g(\mathbf{x}_i)$
- Choice of loss function

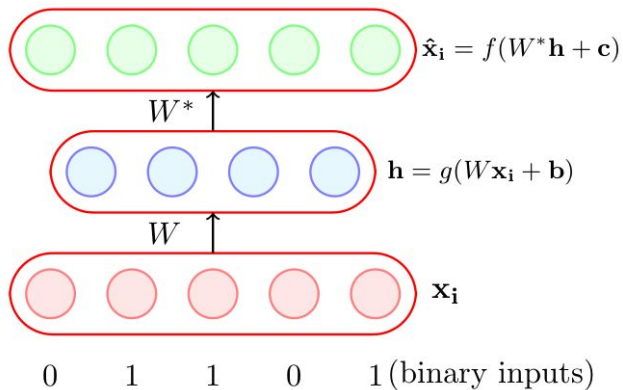
The Road Ahead

- Choice of $f(\mathbf{x}_i)$ and $g(\mathbf{x}_i)$
- Choice of loss function

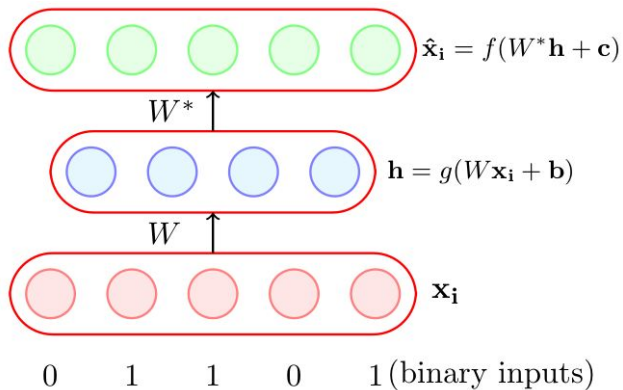


- Suppose all our inputs are binary
(each $x_{ij} \in \{0, 1\}$)



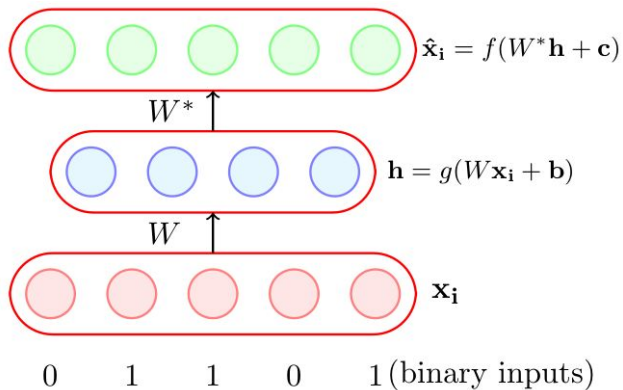


- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?



- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?

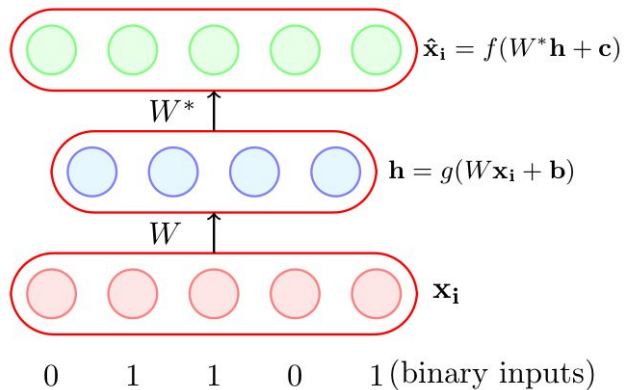
$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$



- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

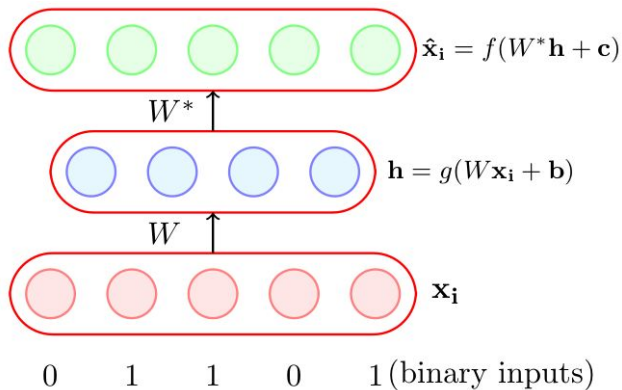


- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^*\mathbf{h} + \mathbf{c})$$



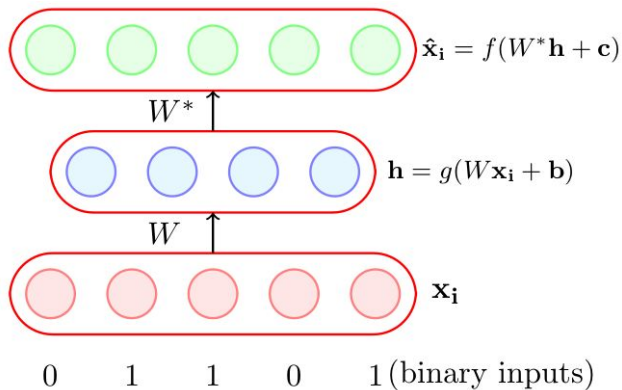
- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^*\mathbf{h} + \mathbf{c})$$

- Logistic as it naturally restricts all outputs to be between 0 and 1



g is typically chosen as the sigmoid function

- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

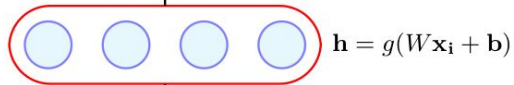
$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^*\mathbf{h} + \mathbf{c})$$

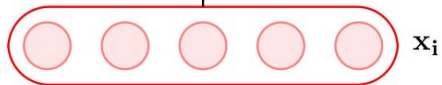
- Logistic as it naturally restricts all outputs to be between 0 and 1



W^*



W



0.25 0.5 1.25 3.5 4.5

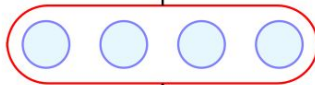
(real valued inputs)

- Suppose all our inputs are real (each



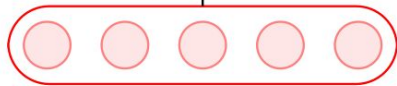
$$\hat{\mathbf{x}}_i = f(W^* \mathbf{h} + \mathbf{c}) \quad x_{ij} \in \mathbb{R}$$

W^* ↑



$$\mathbf{h} = g(W \mathbf{x}_i + \mathbf{b})$$

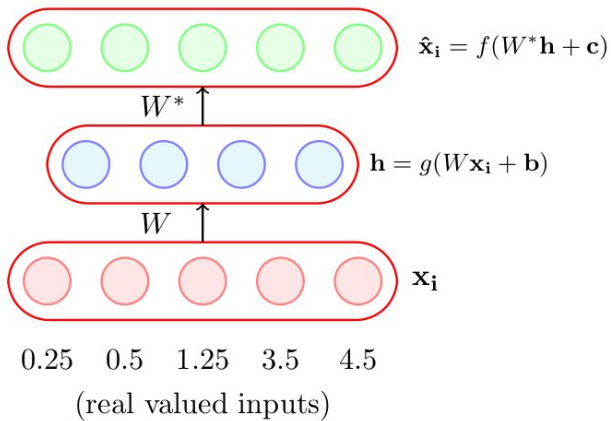
W ↑



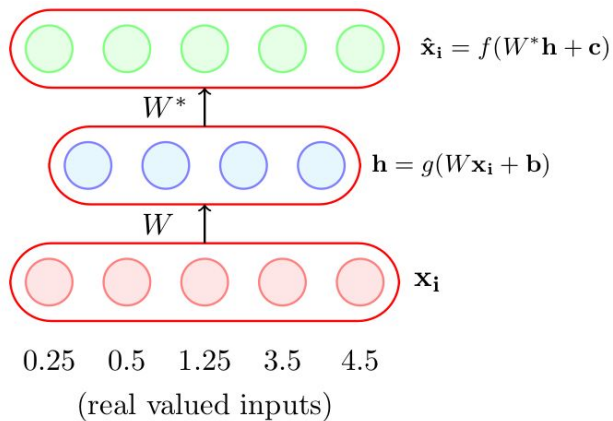
$\mathbf{x}_i$

0.25 0.5 1.25 3.5 4.5

(real valued inputs)

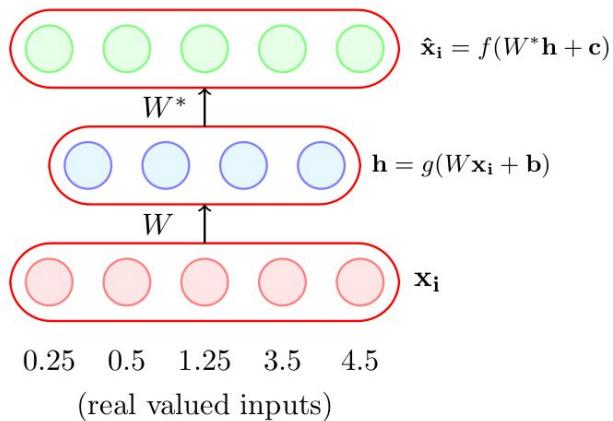


- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?



- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

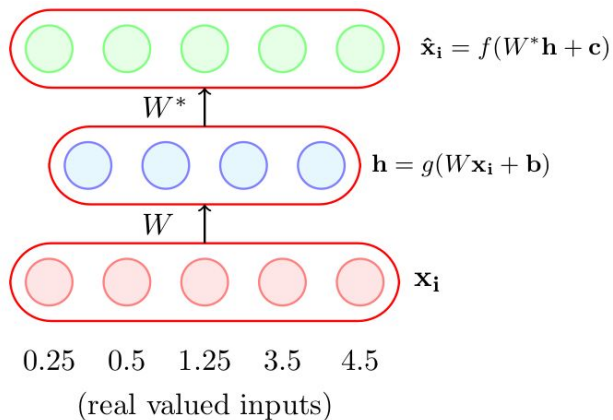
$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$



- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

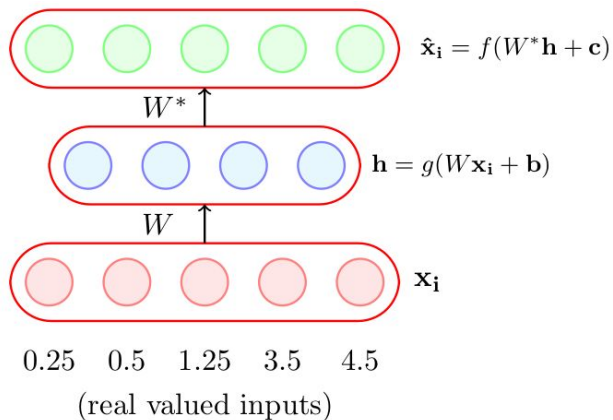


- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^*\mathbf{h} + \mathbf{c})$$



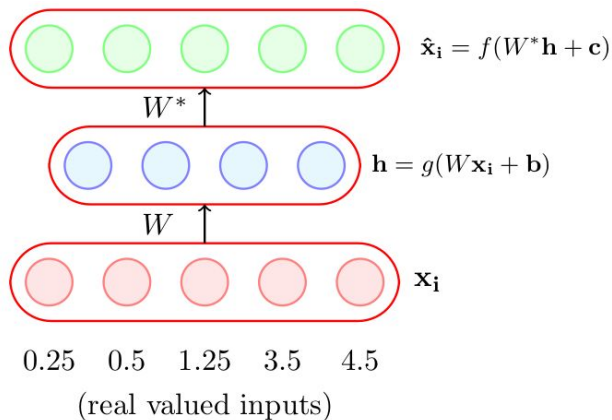
- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{x}_i = \tanh(W^*h + c)$$

$$\hat{x}_i = W^*h + c$$

$$\hat{x}_i = \text{logistic}(W^*h + c)$$

- What will logistic and tanh do?



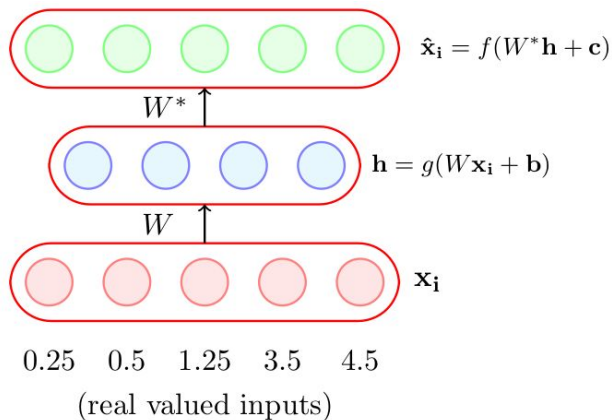
- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^* \mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^* \mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^* \mathbf{h} + \mathbf{c})$$

- What will logistic and tanh do?
- They will restrict the reconstructed $\hat{\mathbf{x}}_i$ to lie between $[0,1]$ or $[-1,1]$ whereas we want $\hat{\mathbf{x}}_i \in \mathbb{R}^n$



Again, g is typically chosen as the sigmoid function

- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^* \mathbf{h} + \mathbf{c})$$

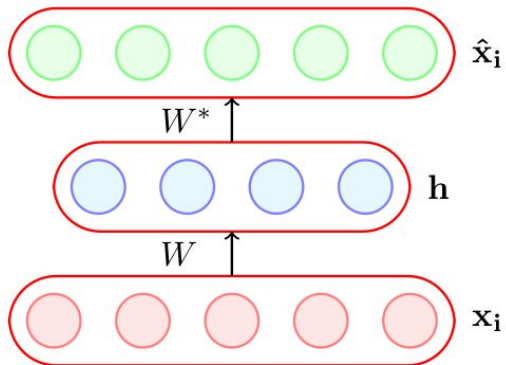
$$\hat{\mathbf{x}}_i = W^* \mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^* \mathbf{h} + \mathbf{c})$$

- What will logistic and tanh do?
- They will restrict the reconstructed $\hat{\mathbf{x}}_i$ to lie between $[0,1]$ or $[-1,1]$ whereas we want $\hat{\mathbf{x}}_i \in \mathbb{R}^n$

The Road Ahead

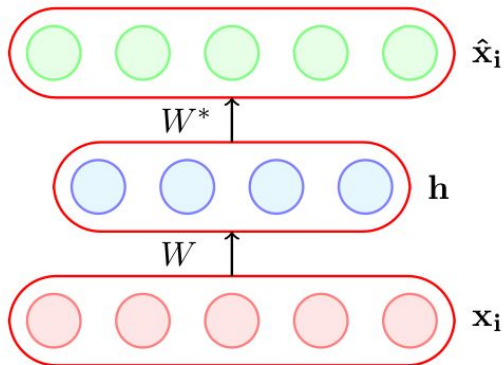
- Choice of $f(\mathbf{x}_i)$ and $g(\mathbf{x}_i)$
- Choice of loss function



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

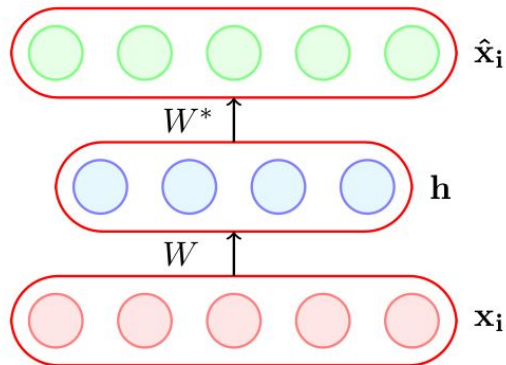
$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Consider the case when the inputs are real valued



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

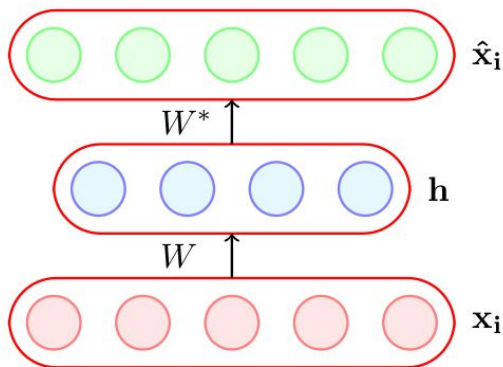
$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ to be as close to $\mathbf{x}_i$ as possible

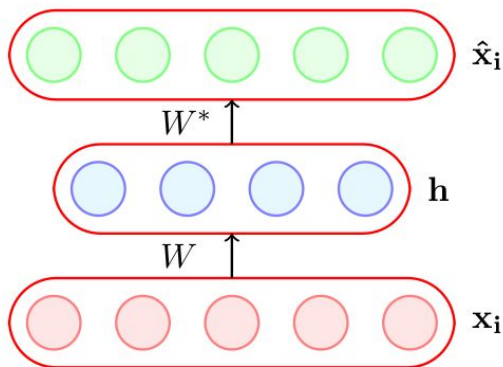


$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ to be as close to $\mathbf{x}_i$ as possible
- This can be formalized using the following objective function:

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$



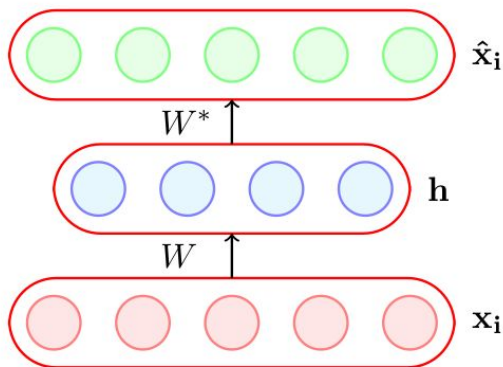
$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ to be as close to $\mathbf{x}_i$ as possible
- This can be formalized using the following objective function:

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

$$i.e., \min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

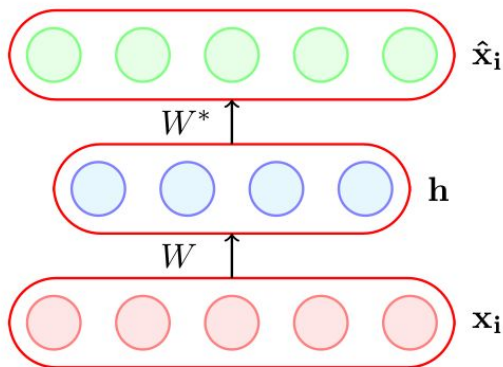
- *there are m instances*
- *each $\mathbf{x}_i$ is n -dimensional*

- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ to be as close to $\mathbf{x}_i$ as possible
- This can be formalized using the following objective function:

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

$$i.e., \min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

- We can then train the autoencoder just like a regular feedforward network using backpropagation



$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b})$$

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

- All we need is a formula for $\frac{\partial \mathcal{L}(\theta)}{\partial W^*}$ and $\frac{\partial \mathcal{L}(\theta)}{\partial W}$

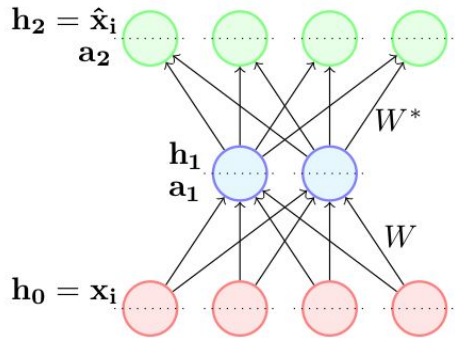
- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ to be as close to $\mathbf{x}_i$ as possible
- This can be formalized using the following objective function:

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

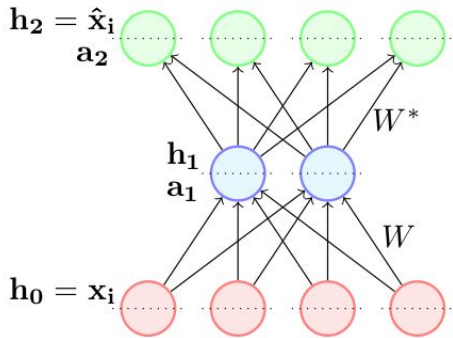
$$i.e., \min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

- We can then train the autoencoder just like a regular feedforward network using backpropagation

$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



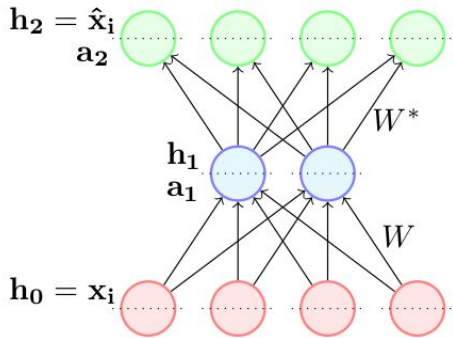
$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



- Note that the loss function is shown for only one training example.

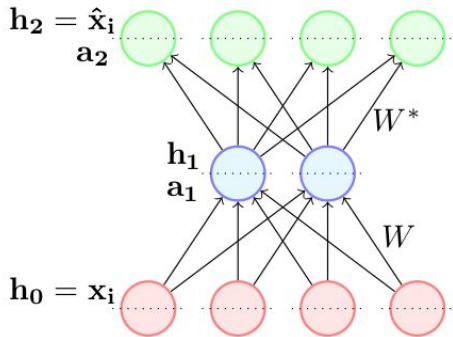
$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

$$\blacksquare \frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial W^*}}$$



- Note that the loss function is shown for only one training example.

$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

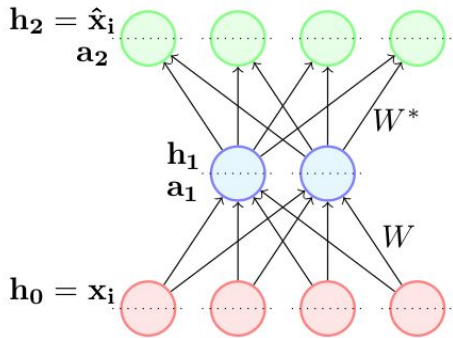


$$\blacksquare \quad \frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial W^*}}$$

$$\blacksquare \quad \frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$$

- Note that the loss function is shown for only one training example.

$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



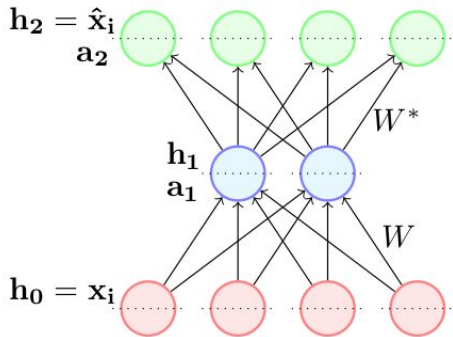
$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \boxed{\frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial W^*}}$$

$$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \boxed{\frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial W}}$$

- expression in the boxes can be calculated using backpropagation

- Note that the loss function is shown for only one training example.

$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



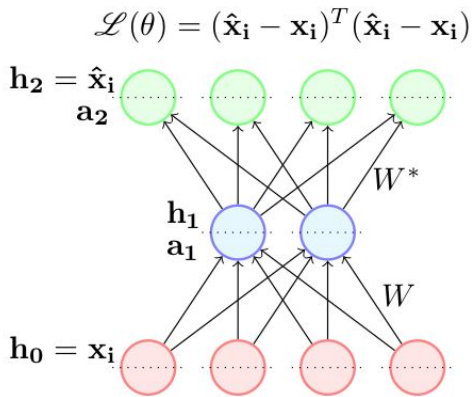
$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial W^*}}$$

$$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$$

- Expression in the boxes are calculated through backpropagation

$$\frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} = \frac{\partial \mathcal{L}(\theta)}{\partial \hat{\mathbf{x}}_i}$$

- Note that the loss function is shown for only one training example.



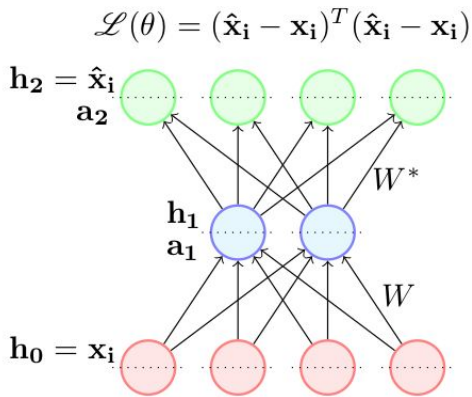
- Note that the loss function is shown for only one training example.

- $$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial W^*}}$$

- $$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$$

- We have already seen how to calculate the expression in the boxes when we learnt backpropagation

$$\begin{aligned} \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} &= \frac{\partial \mathcal{L}(\theta)}{\partial \hat{\mathbf{x}}_i} \\ &= \nabla_{\hat{\mathbf{x}}_i} \{(\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)\} \end{aligned}$$



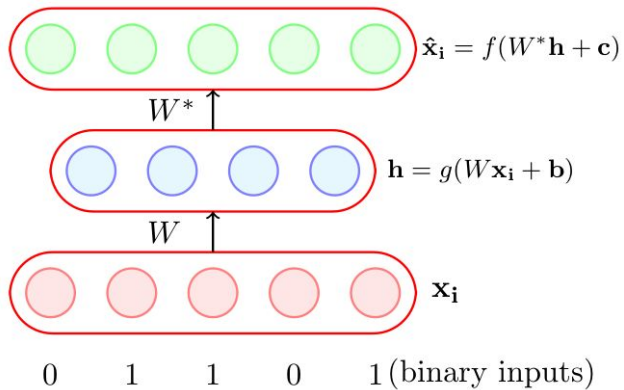
- Note that the loss function is shown for only one training example.

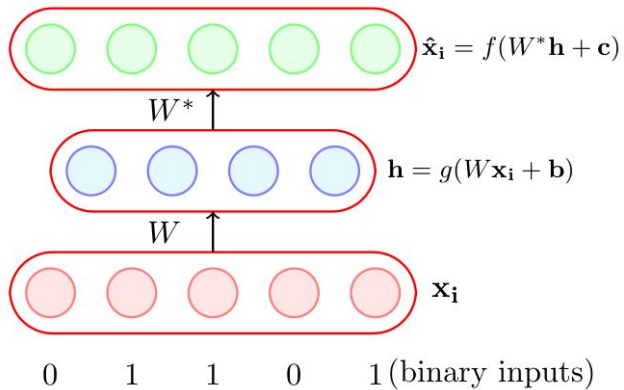
- $$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial W^*}}$$

- $$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$$

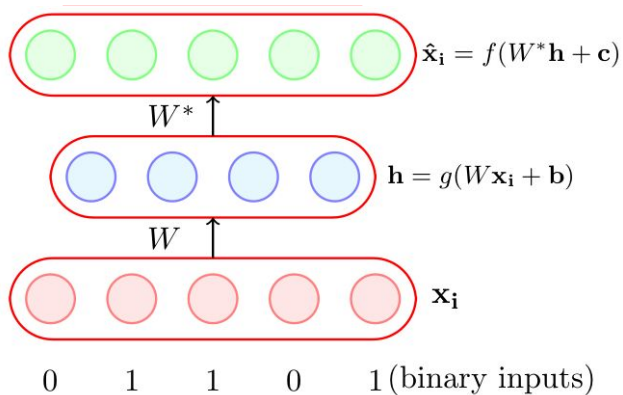
- We have already seen how to calculate the expression in the boxes when we learnt backpropagation

$$\begin{aligned} \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} &= \frac{\partial \mathcal{L}(\theta)}{\partial \hat{\mathbf{x}}_i} \\ &= \nabla_{\hat{\mathbf{x}}_i} \{(\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)\} \\ &= 2(\hat{\mathbf{x}}_i - \mathbf{x}_i) \end{aligned}$$

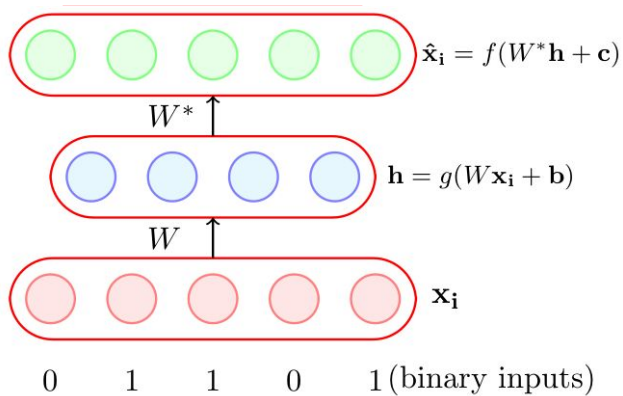




- Consider the case when the inputs are binary

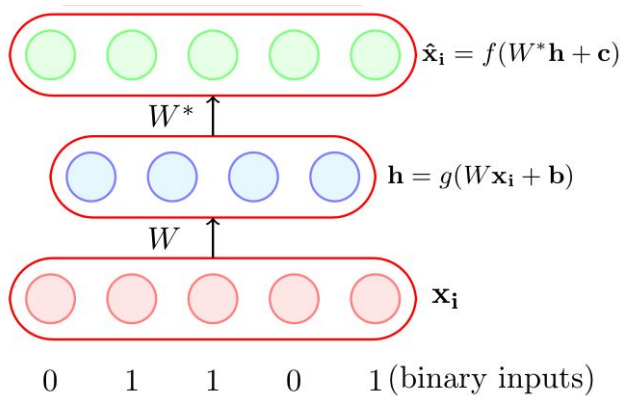


- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.



- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

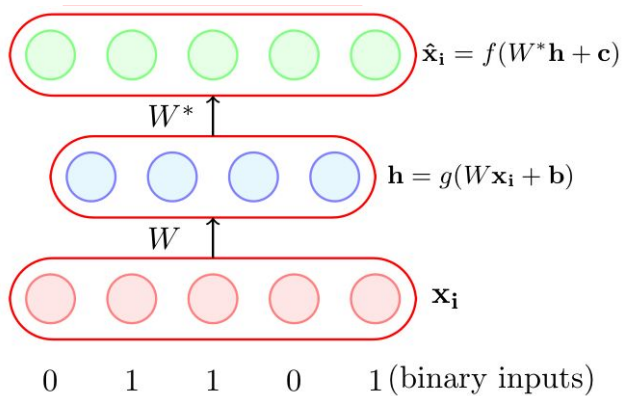
$$\min\left\{-\sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))\right\}$$



- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

$$\min\left\{-\sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))\right\}$$

What value of $\hat{\mathbf{x}}_i$ will minimize this function?

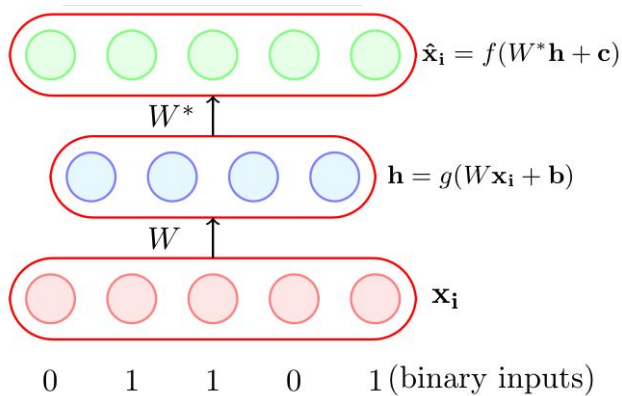


- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

$$\min\left\{-\sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))\right\}$$

What value of $\hat{\mathbf{x}}_i$ will minimize this function?

- If $x_{ij} = 1$?

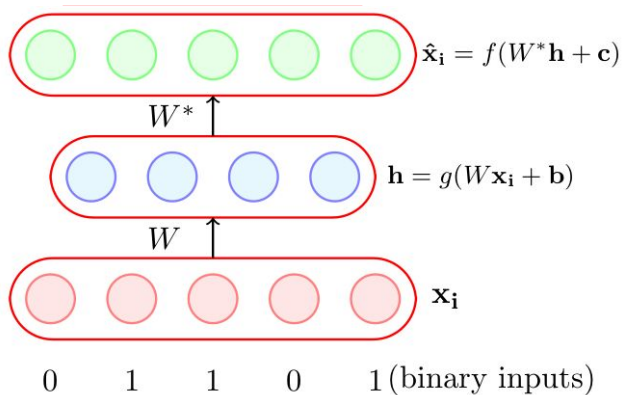


- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

$$\min\left\{-\sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))\right\}$$

What value of $\hat{\mathbf{x}}_i$ will minimize this function?

- If $x_{ij} = 1$?
- If $x_{ij} = 0$?



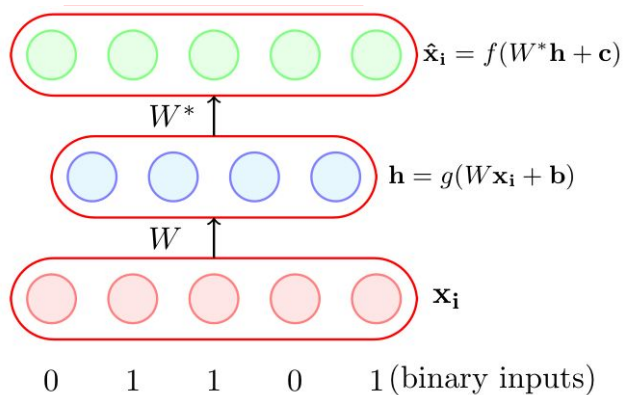
What value of $\hat{\mathbf{x}}_i$ will minimize this function?

- If $x_{ij} = 1$?
- If $x_{ij} = 0$?

- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

$$\min\left\{-\sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))\right\}$$

- Again we need formula for $\frac{\partial \mathcal{L}(\theta)}{\partial W^*}$ and $\frac{\partial \mathcal{L}(\theta)}{\partial W}$ to use back propagation



What value of $\hat{\mathbf{x}}_i$ will minimize this function?

- If $x_{ij} = 1$?
- If $x_{ij} = 0$?

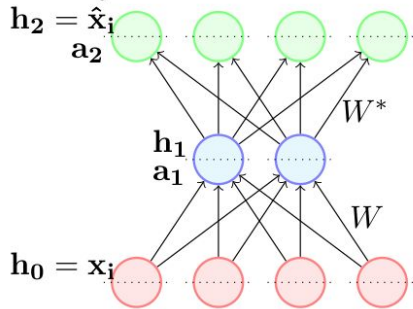
Indeed the above function will be minimized when $\hat{\mathbf{x}}_i = \mathbf{x}_{ij}$!

- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

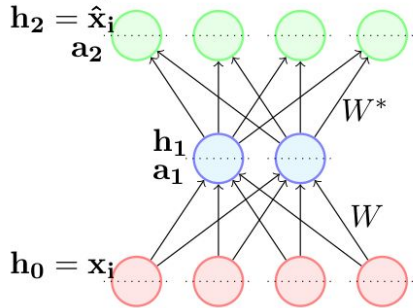
$$\min \left\{ - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log (1 - \hat{x}_{ij})) \right\}$$

- Again we need formula for $\frac{\partial \mathcal{L}(\theta)}{\partial W^*}$ and $\frac{\partial \mathcal{L}(\theta)}{\partial W}$ to use back propagation

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$

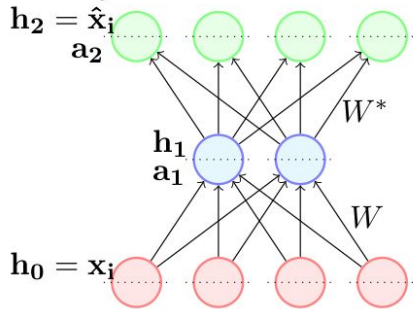


$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



■
$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial W^*}}$$

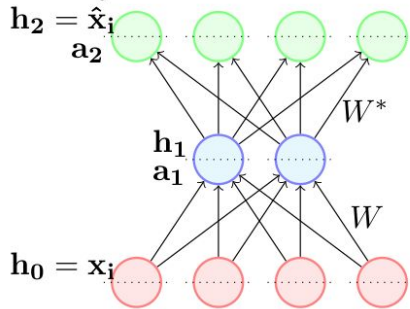
$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial W^*}}$$

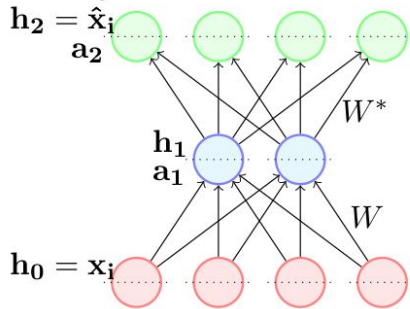
$$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$$

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



- $\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial W^*}}$
- $\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$
- We have already seen how to calculate the expressions in the square boxes when we learnt BP

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$

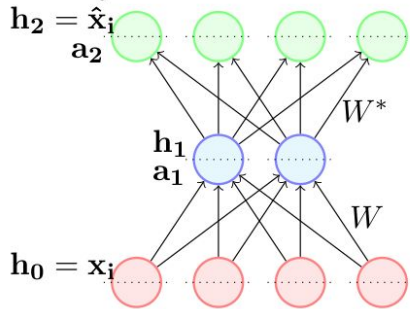


- $\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial W^*}}$
- $\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$
- We have already seen how to calculate the expressions in the square boxes when we learnt BP
- The first two terms on RHS can be computed as:

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_{2j}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}$$

$$\frac{\partial h_{2j}}{\partial a_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$$

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



$$\frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} = \begin{pmatrix} \frac{\partial \mathcal{L}(\theta)}{\partial h_{21}} \\ \frac{\partial \mathcal{L}(\theta)}{\partial h_{22}} \\ \vdots \\ \frac{\partial \mathcal{L}(\theta)}{\partial h_{2n}} \end{pmatrix}$$

- $\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial W^*}}$
- $\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$
- We have already seen how to calculate the expressions in the square boxes when we learnt BP
- The first two terms on RHS can be computed as:

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_{2j}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}$$

$$\frac{\partial h_{2j}}{\partial a_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$$

Acknowledgement:

Deep Learning Course, IITM

Deep Learning for NLP, Stanford University

Thank you for Your Attention